

## **7.1 Introduction to Complexity Theory**

Complexity theory focuses to measure the problems that are decidable or computable in terms of space and time.

### **(a) Time Complexity**

It is a measure of how long a computation takes to execute. In terms of Turing Machine (TM), it is the number of moves made to perform a computation. In terms of digital computer, it is the number of machine cycles required to compute a problem

### **(b) Space Complexity**

It is a measure of how much space is required to compute a problem. In case of TM, it is the measure of tape squares used for a computation. In case of digital computer, it is the number of bytes required to compute a problem.

## 7.2 Order Statistic (The Big O)

*Example:* Given a set of numbers  $x_1, x_2, \dots, x_n$  and a key number  $x$ , determine whether the set contains  $x$  ?

The simplest solution to this problem is the *linear search* from first element to last. That means, it may be found at very beginning or at the very end. The worst case is to compare  $x$  elements of the with  $x$ . In this case we can say the time complexity of linear search is  $O(n)$ .

*Example:* If an algorithm takes exactly  $4n^3 + n^2 + 5n + 56$  machine cycles, where  $n$  is the size of the input, then we can simplify this to  $O(n^3)$ .

### 7.3 Polynomial-Time Algorithms

A polynomial-Time algorithm is an algorithm whose execution time is either given by a polynomial on the size of input, or can be bounded by such a polynomial i.e.,  $T(n) = O(n^k)$  for some constant  $k$ .

*For example:* Bubble sort takes linear time in the least case but it takes  $O(n^2)$  time in the average and worst case. Heapsort takes  $O(n \log n)$  time in all cases.

#### **Definition**

A language  $L$  is in  $P$  if and only if there exists a deterministic Turing machine  $M$ , such that

→  $M$  runs for polynomial time on all inputs.

→ For all  $x$  in  $L$ ,  $M$  outputs *true*.

→ For all  $x$  not in  $L$ ,  $M$  outputs *false*.

## **7.4 Non-Deterministic Polynomial Time Algorithms and Examples**

A Non-Deterministic Polynomial Time Algorithm is the problem that can be computed or executed in polynomial time on a non-deterministic machine. The algorithm either solves the problems in a constant time or it can generate an arbitrary large number of parallel processes.

*Example:* Summary to common time complexities

1.  $O(1)$ : constant time and feasible
2.  $O(\log n)$ : log time and feasible
3.  $O(n)$ : linear time and feasible
4.  $O(n \log n)$ : log linear time and feasible
5.  $O(n^2)$ : quadratic time and sometime feasible
6.  $O(n^3)$ : cubic time and less often feasible
7.  $O(2^n)$ : exponential time and rarely feasible

**7.5 NP-Complete Problems**

A language  $L$  is in class NP if there is a polynomially bounded non-deterministic Turing machine TM such that TM is of time complexity  $P(n)$  for some polynomial  $P$ .

A language  $L$  is said to be NP-Complete if  $L \in NP$  and every  $L_1 \in NP$  is polynomial time reducible to  $L$ . NP complete problems are reducible to one another. This means, given two NP problems  $x$  and  $y$ ,

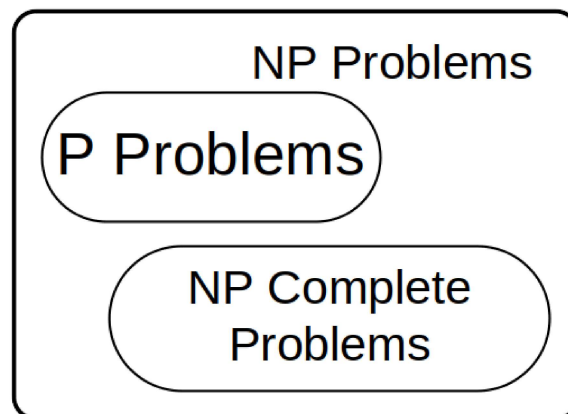
(a) There exists a polynomial time algorithm to restate a problem of type  $x$  as a problem of type  $y$ .

(b) There exists a polynomial time algorithm to translate a solution to a type  $y$  problem back into a solution for the type  $x$  problem.

This is what *complete* means in NP-Complete problems i.e. is there exist any polynomial-time algorithm for any of these problems (which derives polynomial time algorithm for all of them)? This leads to:

Does  $P = NP$  ?

No one has ever found a deterministic polynomial time algorithm for of these problems and also no one has ever succeeded in proving that no deterministic polynomial time algorithm exists either. Now a days, most computer scientists don't think polynomial-time algorithm can exist, but no one knows for sure.



## 7.6 Tractable and Intractable Problems,

Those problems which we can solve using polynomial time algorithm are *tractable* problems otherwise it would be intractable (*exponential-time solution on a deterministic machine*) problems. For example: Boolean Satisfiability problem, Integer Bin Packing problem, Travelling salesman problem etc.

## 7.7 Integer Bin Packing Problems

Assume that we are given a set of  $n$  integers. Our task is to arrange these integers into two ***bins or piles*** so that the sum of the integers in one pile is equal to the sum of the integers in the other pile.

A general solution (deterministic) to this problems are:

For example:  $\{19, 23, 32, 42, 50, 62, 77, 88, 89, 105, 114, 123, 176\}$  numbers sum to 1000. Can we divide the set into two groups and the sum of the elements in individual group is 500 ?

This is a deterministic algorithm ( $2^n$  possible combinations) and we can represent the set by 13 bits number. If we iterate  $i$  ranging from 1 to 13 and number put 1's position at bin 1 and rest put to the another bin. Finally we will get the solution if exists.



**7.8 Boolean Satisfiability Problems**

Assume that we have  $n$  Boolean variables such as  $A, B, C, \dots$  and an Boolean expression made from the variables then, Is there an assignment of truth values to the variables gives the expression true ?

*Example:*  $\phi = (\bar{x} \wedge y) \vee (x \wedge \bar{z})$

If we put  $x = 0, y = 1$  and  $z = 0$ , we will get  $\phi = 1$ . This is Boolean satisfiable problem. ( $2^n$  possible assignments)

## **7.9 Travelling Salesman Problems (TSP)**

A salesman starting in Texas, wants to visit every capital city in the US, returning to Texas as his last stop. In what order should he visit the capital cities so as to *minimize the total distance travelled* ?

### **7.10 Linear Programming Problem**

We have on hand  $x$  amount of butter,  $y$  amount of flour,  $z$  eggs etc. We have cookie recipes that use varying amounts of these ingredients. Different kinds of cookies bring different prices. What mix of cookies should we make in order to maximize profits ?

### **7.11 The Hamiltonian Circuit Problem**

Every capital city has direct air flights to at least some capital cities. Our intrepid salesman wants to visit all the capitals, and return to his starting point, taking only direct flight. Can he find a path that lets him do this ?